



FACULTAD REGIONAL ROSARIO

Cátedra de: ENTORNOS GRÁFICOS
"Conceptos Fundamentales para la Generación de Presupuestos"

Prof. Ing. Daniela E. Díaz
Prof. Ing. Julián Butti
UTN-FRRO



Presupuestos

Para elaborar el presupuesto de un proyecto de software, lo primero que necesitaremos será entender en qué consiste el software a desarrollar y cuáles serán sus funcionalidades.

Seguidamente necesitaremos medir la magnitud del trabajo a realizar y determinar el costo por unidad de medida, llegando así a un costo por funcionalidad y total.

Tengamos en cuenta, que este tipo de estimaciones, son justamente, una estimación imprecisa, o un modelo de costos que pretendemos cubrir, pero que en general tiene desvíos. Las estimaciones se pueden volver más precisas con la experiencia y el conocimiento, además de saber de antemano el resultado esperado del proyecto.

Para desarrollar el ejemplo de presupuesto de un proyecto de software, seguiremos los siguientes pasos:

- Análisis de los requerimientos del proyecto de desarrollo de software
- Medición del software.
- Determinación de la productividad del equipo de trabajo.
- Estimación de esfuerzo y personal necesarios para el presupuesto.
- Costos del personal.
- Costos de las unidades de medida del proyecto.
- Presupuesto de un proyecto de software

Análisis de requerimientos

El primer paso es obtener un entendimiento de que es lo que necesita el cliente. Para ello, debemos realizar una serie de entrevistas utilizando técnicas de levantamiento de información de requerimientos de software.

Supongamos que para el ejemplo de presupuesto de un proyecto de software que estamos desarrollando, realizamos el levantamiento de información y hemos determinado que los requerimientos son 6 casos de uso principales. Vamos a registrarlos como casos de uso, requerimientos funcionales, historias de usuarios, según sea el método elegido y preferencias.

Mientras más descriptivos y precisos, menos será el margen de error. Tener en cuenta los casos alternativos, validaciones, etc.

Con los requerimientos funcionales definidos podemos seguir con el siguiente paso, que será realizar una medición de la magnitud o tamaño del software a desarrollar.



Medición del software

Existen diversas técnicas de estimación de esfuerzo y costo en software a nuestra disposición, para este ejemplo, utilizaremos como método de medición el Análisis de puntos de función definido por el International Function Points Group (IFPUG).

Desarrollaremos la medición en dos pasos, primero determinaremos los componentes funcionales del presupuesto de desarrollo de software, a partir del análisis de requerimientos realizado anteriormente. Seguidamente, realizaremos el cálculo de los puntos de función, con lo cual obtendremos una medida del tamaño del proyecto.

Una de las ventajas de las métricas de software usando el método de puntos de función, es que al multiplicar el tamaño funcional de un sistema por el costo de producir cada punto de función, quienes toman las decisiones en la organización pueden realizar cálculos rápidos del costo de desarrollo de un software y por lo tanto de su presupuesto.

Otra alternativa, es estimar por puntos de complejidad. Por ejemplo, una tarea se le asigna un puntaje usando la escala de scrum sugerida, 1,3,5,7,13, según el grado de complejidad. Una tarea muy sencilla tendrá un peso de 1 punto, y una tarea muy compleja podría tener asignado el 13. Si hay muchas tareas con 13 puntos, se torna evidente que tendríamos que particionarla en varias tareas de menor puntaje. De esta manera podemos afinar el proceso y aminorar el margen de error en la estimación.

También es importante, mantener esta lista de requerimientos, ordenadas por prioridad, de manera que hagamos foco en lo más importante a resolver desde el inicio.

Suponemos ahora, que obtenemos un listado con la siguiente forma:

- REQ01 - Requerimiento 1 (Prioridad) - Descripción - X puntos.
- REQ02 - Requerimiento 2 (Prioridad) - Descripción - X puntos.
- REQ03 - Requerimiento 3 (Prioridad) - Descripción - X puntos.
- REQ04 - Requerimiento 4 (Prioridad) - Descripción - X puntos.

Y obtenemos una sumatoria de 72 puntos, ya sea de función, de complejidad scrum, o de la técnica que utilicemos.

Productividad del equipo de trabajo

Ahora necesitamos conocer cuántos puntos puede desarrollar nuestro equipo de trabajo en un tiempo dado, esto para determinar la duración del proyecto.

Necesitamos valernos de la información de proyectos pasados que tenga la organización, también podemos usar información de otras fuentes, otras organizaciones y bases de datos, recomendaciones, etc.



Por ejemplo, supongamos que sabemos que un equipo de trabajo compuesto por 4 desarrolladores, 2 testers y un director de proyectos, pueden producir 30 puntos al mes.

Para que la medición sea exacta, debemos considerar puntos de función completamente desarrollados, es decir donde se ejecutó el análisis, diseño, desarrollo, pruebas. Es decir, todo lo necesario para que sea puesto en producción.

Determinar la productividad solamente con tiempos de desarrollo no nos ayuda en este caso.

Considerando que un mes tiene 20 días laborables en promedio, podemos determinar el número de jornadas que necesitamos para producir 30 puntos;

Para producir 30 puntos necesitamos:

- 60 días de desarrolladores de software.
- 20 días de testing.
- 10 días de Project Management.

Por lo tanto la productividad del equipo es:

- Desarrollo de software: 0,5 puntos por día.
- Testing: 1,5 puntos por día.
- PM: 3 puntos por día.

Estimación de esfuerzo y personal necesarios en base a la productividad

Ahora que sabemos la productividad, podremos realizar el cálculo del esfuerzo necesario (medido en días) y estimación de personal.

En el ejemplo, sabiendo que el desarrollo de software tiene una medición de 72 puntos, podemos determinar que necesitamos:

- Desarrolladores de software: 144 días.
- Testers: 48 días.
- PM: 24 días.

Para definir el tiempo de ejecución, tenemos que tener en cuenta la cantidad de personas de nuestro equipo, y puede variar a lo largo del proyecto.

Podríamos aumentar el número de personas para ejecutarlo en menos tiempo, sin embargo, no podemos aplicar la misma productividad que calculamos en base al equipo de trabajo previo, pues está demostrado que la relación cantidad de personas con la productividad no es linealmente proporcional.



Para evitar este tipo de variantes de productividad, se puede también, como alternativa, ajustar la estimación y realizar los cálculos, en base a horas de esfuerzo. Más allá del tamaño del equipo, tendremos una magnitud de la cantidad de horas destinadas a dicho esfuerzo.

Otra opción a tener en cuenta, si no sabemos la productividad, o no tenemos un equipo completamente formado, es estimar las horas de desarrollo, y sobre el subtotal, asignar un porcentaje de testing y gestión, además de un porcentaje de ganancia sobre el proyecto.

Por ejemplo, si estimamos que llevaría 200 horas de esfuerzo de programación, podríamos asignar un 20% adicional al testing, y un 10% adicional de PM. A este subtotal, de 260 horas, podríamos además agregarle un margen de ganancia o de emergencia ante una mala estimación.

Costos del personal del proyecto de desarrollo de software

Teniendo definidas el número de días/horas que requiere nuestro proyecto de desarrollo de software, pasamos a determinar los costos, para lo cual lo primero que necesitamos conocer es el costo del personal.

Para nuestro ejemplo de presupuesto, los costos mensuales de nuestro personal serán los siguientes:

PM: \$ 95.000

Desarrollador de software: \$ 60.000

Tester: \$ 35.000

Para los precios/sueldos, pueden tomar como referencia el siguiente sitio:

<https://cessi.org.ar/opssi/> Observatorio de la industria del Software o

<https://www.cpcipc.org.ar/content/honorarios> del Consejo Profesional de Ciencias Informáticas de la Provincia de Córdoba.

Con estos datos podríamos determinar el costo por día/hora, y como conocemos las jornadas del proyecto podríamos calcular el presupuesto. Sin embargo, para obtener una medida de costo que podamos aplicar universalmente a todos los proyectos, es más útil definir cuál es el costo de un punto.

Costos de las unidades de medida del proyecto (Puntos)

Tomando los costos de personal, el costo mensual del equipo de desarrollo de software es el siguiente:

PM (1 persona): \$ 95.000

Desarrollador de software (4 personas): \$ 240.000

Tester (2 personas): \$ 70.000

Total: \$ 405.000



En un mes de trabajo el equipo puede desarrollar 30 puntos de función, por lo tanto el costo por unidad de medida es:

$\text{Costo por unidad de medida} = \text{Costo total} / \text{Nro. De unidades de medida}$

$\text{Costo por unidad de medida} = \$ 405.000 / 30 \text{ puntos} = \$ 13.500 \text{ por punto.}$

Presupuesto del proyecto de desarrollo de software

Sabiendo que nuestro proyecto tiene 72 puntos de función, podemos estimar su costo en:

$\$ 13.500 \times 72 \text{ puntos de función} = \$972.000.-$

De hecho, dado que conocemos los puntos de cada componente funcional del proyecto, podríamos inclusive presentar un presupuesto desglosado, asignando un valor a cada ítem.

Tener en cuenta también las opciones de pago que se le brindan al cliente. No es lo mismo recibir un pago adelantado por el total, que una larga financiación. Para ello tener en cuenta las condiciones y contexto económico financiero del equipo, el país, el cliente, etc.

Otras consideraciones

Técnicas de Estimación

1. Puntos de Función (FPUG)
 - Componentes funcionales
 - Cálculo de puntos
 - Ventajas para decisiones organizacionales
2. Puntos de Complejidad (Scrum)
 - Escala: 1, 3, 5, 7, 13
 - Criterios de asignación
 - Particionamiento de tareas complejas

Métodos Modernos de Estimación

1. Planning Poker
 - Técnica grupal colaborativa
 - Uso de cartas con valores Fibonacci
 - Discusión de estimaciones divergentes
2. T-Shirt Sizing
 - Categorías: XS, S, M, L, XL
 - Útil para estimaciones iniciales



- Conversión posterior a puntos
- 3. Estimación Paramétrica
 - Basada en datos históricos
 - Fórmulas y modelos matemáticos
 - Ajustes por complejidad

Productividad del Equipo

Métricas Base

- 30 puntos mensuales por equipo estándar
- 20 días laborables promedio
- Distribución de esfuerzo por rol

Productividad por Rol

- Desarrollo: 0.5 puntos/día
- Testing: 1.5 puntos/día
- PM: 3 puntos/día

Estimación de Esfuerzo y Personal

Cálculo Base

- Desarrollo: 144 días
- Testing: 48 días
- PM: 24 días

Consideraciones de Escalado

- Relación no lineal entre personal y productividad
- Ajustes por tamaño de equipo
- Estimación por horas de esfuerzo

Costos Adicionales

1. Infraestructura
 - Servidores y hosting
 - Licencias de software
 - Herramientas de desarrollo
2. Capacitación
 - Entrenamiento técnico
 - Workshops
 - Documentación
3. Soporte Post-implementación
 - Mantenimiento



- Actualizaciones
- Soporte técnico
- 4. Costos Indirectos
 - Oficina y equipamiento
 - Servicios básicos
 - Seguros

Contingencias y Gestión de Riesgos

En el ámbito del desarrollo de software, la **gestión de riesgos** es un proceso esencial para identificar, analizar y mitigar posibles problemas que puedan afectar el éxito de un proyecto.

Las **contingencias**, por su parte, son planes de acción diseñados para responder de manera efectiva ante la materialización de dichos riesgos.

Juntos, estos conceptos forman parte de una estrategia proactiva para garantizar que los proyectos se completen dentro del plazo, presupuesto y calidad esperados.

¿Qué es un Riesgo en el Desarrollo de Software?

Un **riesgo** es cualquier evento o condición incierta que, de ocurrir, podría tener un impacto negativo en el proyecto.

Estos riesgos pueden estar relacionados con:

- **Factores técnicos:** Fallos en la tecnología, dependencia de herramientas obsoletas o dificultades en la integración de sistemas.
- **Factores humanos:** Rotación de personal, falta de habilidades técnicas o problemas de comunicación en el equipo.
- **Factores organizacionales:** Cambios en los requisitos del cliente, recortes presupuestarios o prioridades cambiantes.
- **Factores externos:** Cambios en regulaciones, problemas con proveedores o eventos imprevistos (como desastres naturales).
-

Proceso de Gestión de Riesgos

La **gestión de riesgos** es un proceso continuo que se lleva a cabo a lo largo de todo el ciclo de vida del proyecto. Incluye las siguientes etapas:

1. Identificación de Riesgos:

Se detectan y documentan los posibles riesgos que podrían afectar al proyecto. Esto se logra mediante técnicas como **brainstorming** (el **brainstorming**, también conocido como **lluvia de ideas**, es una técnica creativa utilizada para generar una gran cantidad de ideas o soluciones en un corto período de tiempo; es una herramienta muy popular en entornos colaborativos, como equipos de trabajo, proyectos creativos, desarrollo de software, marketing y muchas otras áreas. Su objetivo principal es fomentar la participación libre y abierta de todos los miembros del grupo, sin restricciones ni juicios, para explorar nuevas posibilidades y enfoques), entrevistas con **stakeholders** o análisis de proyectos anteriores.



2. **Análisis de Riesgos:** Se evalúa la probabilidad de que ocurra cada riesgo y su impacto potencial. Esto permite priorizar los riesgos y enfocar los esfuerzos en los más críticos.
3. **Planificación de Respuestas:** Se definen estrategias para abordar los riesgos. Estas pueden incluir:
 - Evitación: Modificar el proyecto para eliminar el riesgo.
 - Mitigación: Reducir la probabilidad o el impacto del riesgo.
 - Transferencia: Trasladar el riesgo a un tercero (por ejemplo, mediante un contrato con un proveedor).
 - Aceptación: Asumir el riesgo si su impacto es bajo o si no es viable evitarlo.
4. **Monitoreo y Control:** Durante la ejecución del proyecto, se supervisan los riesgos identificados y se detectan nuevos riesgos. Esto permite ajustar los planes de contingencia según sea necesario.

3. Planes de Contingencia

Un plan de contingencia es una respuesta predeterminada que se activa cuando un riesgo se materializa. Estos planes deben ser:

- Específicos: Definir acciones claras y concretas.
- Realistas: Ser viables dentro del contexto del proyecto.
- Oportunos: Poder implementarse rápidamente para minimizar el impacto.

Por ejemplo, si un proveedor clave no entrega un componente a tiempo, un plan de contingencia podría ser recurrir a un proveedor alternativo o utilizar una solución temporal mientras se resuelve el problema.

4. Beneficios de la Gestión de Riesgos y Contingencias

- Reducción de incertidumbre: Permite anticipar problemas y tomar decisiones informadas.
- Ahorro de tiempo y costos: Evita retrasos y gastos imprevistos.
- Mejora de la calidad: Al abordar riesgos técnicos, se asegura que el software cumpla con los estándares requeridos.
- Mayor confianza del cliente: Demuestra profesionalismo y capacidad para manejar imprevistos.

La gestión de riesgos y la preparación de contingencias son prácticas fundamentales en el desarrollo de software. No se trata de evitar todos los problemas, sino de estar preparados para enfrentarlos de manera efectiva.

Un enfoque proactivo en la identificación y mitigación de riesgos no solo aumenta las probabilidades de éxito del proyecto, sino que también contribuye a la mejora continua de los procesos de desarrollo.

Buffer de Contingencia

El Buffer de Contingencia (también conocido como reserva de contingencia o margen de seguridad) es una cantidad de tiempo, recursos o presupuesto que se reserva en un proyecto para hacer frente a imprevistos, riesgos o incertidumbres que puedan surgir durante su ejecución. Es una herramienta de gestión de proyectos que ayuda a garantizar que, a pesar de los posibles contratiempos, el proyecto pueda completarse dentro del plazo y presupuesto establecidos.

- 15-30% sobre el presupuesto base
- Factores de ajuste por complejidad



- Reservas para imprevistos

Riesgos Técnicos

- Complejidad tecnológica
- Integración con sistemas existentes
- Seguridad y performance

Cambios de Alcance

- Proceso de control de cambios
- Impacto en cronograma
- Ajustes presupuestarios

Métricas de Control

Son herramientas cuantitativas utilizadas en la gestión de proyectos y procesos para medir, monitorear y evaluar el desempeño, el progreso y la calidad de un proyecto, producto o servicio. Estas métricas proporcionan datos objetivos que permiten a los equipos y gestores tomar decisiones informadas, identificar áreas de mejora y asegurar que los objetivos se estén cumpliendo de manera efectiva.

En el contexto del desarrollo de software, las métricas de control son especialmente importantes para garantizar que el proyecto avanza según lo planeado, que se cumplen los requisitos de calidad y que se gestionan adecuadamente los recursos, el tiempo y los costos.

KPIs de Seguimiento

Las **métricas KPIs de Seguimiento** (Key Performance Indicators, o Indicadores Clave de Desempeño) son herramientas de medición utilizadas para evaluar el rendimiento y el progreso de un proyecto, proceso o actividad en relación con objetivos específicos. Estos indicadores son esenciales para monitorear el éxito de una iniciativa y tomar decisiones basadas en datos. En el contexto de la gestión de proyectos, desarrollo de software o negocios, los KPIs de seguimiento permiten a los equipos y líderes identificar si se están cumpliendo las metas establecidas y, en caso contrario, tomar acciones correctivas.

- Valor ganado (EV)
- Costo real vs presupuestado
- Variación del cronograma

Earned Value Management

Earned Value Management (EVM), o **Gestión del Valor Ganado**, es una metodología de gestión de proyectos que combina mediciones de alcance, cronograma y costos para evaluar el desempeño y el progreso de un proyecto.



Esta técnica permite a los gestores de proyectos determinar si un proyecto está avanzando según lo planeado, si se está cumpliendo con el presupuesto y si se están logrando los objetivos dentro del tiempo establecido.

EVM es ampliamente utilizado en proyectos complejos, especialmente en sectores como construcción, defensa, ingeniería y desarrollo de software, donde el control del tiempo y los costos es crítico.

- CPI (Cost Performance Index)
- SPI (Schedule Performance Index)
- Proyecciones y tendencias

Aspectos Comerciales

Modelos de Facturación

1. Precio Fijo
 - Alcance definido
 - Riesgos y contingencias
 - Términos de pago
2. Tiempo y Materiales
 - Tarifas por rol
 - Facturación periódica
 - Límites de presupuesto

Estrategias de Negociación

- Opciones de alcance
- Planes escalonados
- Términos y condiciones

Checklist de Validación

1. Requerimientos completos y validados
2. Estimaciones revisadas por equipo
3. Riesgos identificados y cuantificados
4. Costos indirectos incluidos
5. Contingencias calculadas
6. Términos comerciales definidos

Links relacionados / de interés:

<https://www.incentro.com/es-es/blog/stories/scrum-puntos-de-historia-como-funcionan/>
<https://lapastillaroja.net/2019/04/como-estimar-el-esfuerzo-de-desarrollo-de-software/>
<http://laboratorios.fi.uba.ar/lsi/rqm/articulos/R-ITBA-22-estimacion.pdf>
<https://cessi.org.ar/opssi/>